

Day 14 Hands-on Assignment

Designing an API Gateway and Service Communication for a Simple E-commerce Application

John Michael Bilbao
Techstart
September 2, 2026

Introduction

Bilbao Bazaar is a small online shop split into two microservices. One owns the catalogue: what is for sale, what it costs, how many are left. The other takes orders. They are separate programs in separate containers, and neither can read the other's data.

That split is why the assignment needs a gateway. A customer should not have to know that products live on one port and orders on another, and the browser should not be making requests to two different origins to draw one page. So a third container sits in front of both and is the only thing anybody talks to. It is an NGINX instance, and it does the routing.

There is a second kind of traffic that has nothing to do with the customer. When an order arrives, the orders service does not know what anything costs, so it has to ask the catalogue. That call goes sideways, service to service, and it turned out to be the more interesting half of the design.

Everything runs on Docker: three images, three containers, one compose file. `run.sh` builds, starts, exercises and tears down the whole stack in one invocation and writes every command and response into `results/`. The figures below are rendered from those captures and from real Chromium screenshots, so nothing here is retyped by hand.

Container	Image	Listens on	What it does
bazaar-gateway	bazaar/api-gateway:1.0	0.0.0.0:8091	NGINX. Routes <code>/api/...</code> to both services, serves the storefront.
bazaar-products	bazaar/products-service:1.0	127.0.0.1:8000	The catalogue. Products, prices, stock. Calls nobody.
bazaar-orders	bazaar/orders-service:1.0	127.0.0.1:8001	Takes orders. Prices every line through products-service.

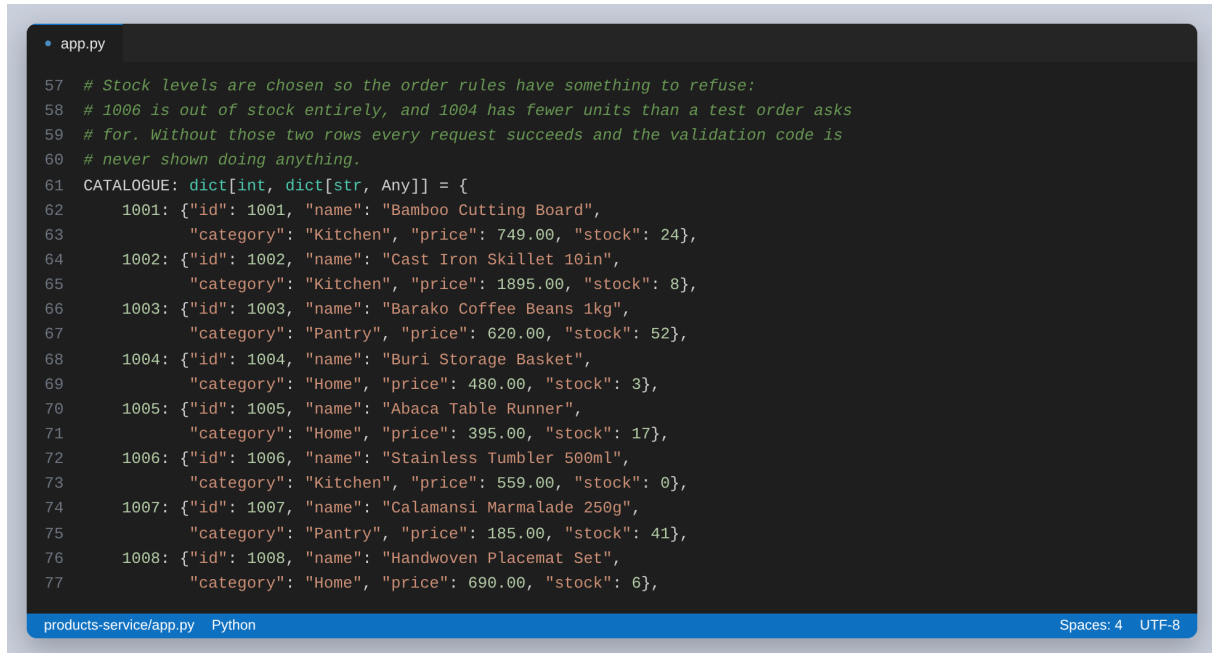
The two services are FastAPI applications on Python 3.12. The gateway is nginx 1.27 on Alpine, which is why its image is 49.7 MB against 171 MB for a service carrying a Python runtime.

A note on the networking, because it is not the shape a textbook would draw. The intended compose file put all three containers on a bridge network and published only the gateway's port. That does not run on the machine I built this on: the container runtime is Podman without `CAP_NET_ADMIN`, so it allocates addresses on the bridge and then programs no routes to them. All three containers share the host network namespace instead, and the isolation is done with bind addresses. The last section explains

how I worked that out.

The two microservices

The catalogue is eight products with an id, a name, a category, a price in pesos and a stock count, held in memory. Two of the rows are chosen rather than invented: product 1006 has zero stock and product 1004 has three units, so there is something for the order rules to refuse. Without them every request succeeds and the validation code never appears in a screenshot.



```
• app.py
57 # Stock levels are chosen so the order rules have something to refuse:
58 # 1006 is out of stock entirely, and 1004 has fewer units than a test order asks
59 # for. Without those two rows every request succeeds and the validation code is
60 # never shown doing anything.
61 CATALOGUE: dict[int, dict[str, Any]] = {
62     1001: {"id": 1001, "name": "Bamboo Cutting Board",
63           "category": "Kitchen", "price": 749.00, "stock": 24},
64     1002: {"id": 1002, "name": "Cast Iron Skillet 10in",
65           "category": "Kitchen", "price": 1895.00, "stock": 8},
66     1003: {"id": 1003, "name": "Barako Coffee Beans 1kg",
67           "category": "Pantry", "price": 620.00, "stock": 52},
68     1004: {"id": 1004, "name": "Buri Storage Basket",
69           "category": "Home", "price": 480.00, "stock": 3},
70     1005: {"id": 1005, "name": "Abaca Table Runner",
71           "category": "Home", "price": 395.00, "stock": 17},
72     1006: {"id": 1006, "name": "Stainless Tumbler 500ml",
73           "category": "Kitchen", "price": 559.00, "stock": 0},
74     1007: {"id": 1007, "name": "Calamansi Marmalade 250g",
75           "category": "Pantry", "price": 185.00, "stock": 41},
76     1008: {"id": 1008, "name": "Handwoven Placemat Set",
77           "category": "Home", "price": 690.00, "stock": 6},

```

Figure 1: The catalogue. Two rows exist to make the order rules do something visible

The endpoint the orders service depends on is the single-product one. It returns a 404 for an id that is not there, and that 404 is what makes an order for a nonexistent product fail with a 404 rather than a 500.

```
• app.py

132 @app.get("/products/{product_id}", tags=["catalogue"])
133 def get_product(product_id: int) -> dict[str, Any]:
134     """One product.
135
136     This is the endpoint the orders service calls. A 404 here is what makes an
137     order for a product that does not exist fail with a 404 rather than a 500.
138     """
139     product = CATALOGUE.get(product_id)
140     if product is None:
141         raise HTTPException(
142             status_code=404,
143             detail=f"no product with id {product_id} in the catalogue")
144     return {"currency": CURRENCY, "served_by": SERVICE, **product}

products-service/app.py Python Spaces: 4 UTF-8
```

Figure 2: GET /products/{id}, the endpoint the orders service calls

The orders service holds no prices at all. It takes a product id and a quantity, asks the catalogue what that product costs, checks the answer against its own rules, and only then writes an order. The address it asks is configuration, never a literal:

```
• app.py

36 # The dependency arrives as configuration, never as a literal in the code. In
37 # compose it is the service name; in another environment it could be a load
38 # balancer, and nothing in this file would change.
39 PRODUCTS_BASE_URL = os.getenv("PRODUCTS_BASE_URL", "http://products-service:8000")
40 MAX_QUANTITY = int(os.getenv("MAX_QUANTITY_PER_ORDER", "10"))
41 # Short on purpose. A dependency that has stopped answering should surface as a
42 # fast 503, not as a request that hangs until the client gives up.
43 TIMEOUT = float(os.getenv("PRODUCTS_TIMEOUT_SECONDS", "3.0"))

orders-service/app.py Python Spaces: 4 UTF-8
```

Figure 3: The dependency and the two limits, all arriving as environment variables

Task 1: Designing the API gateway

I used NGINX rather than a managed gateway because the whole design then fits in one file I can show, and because it runs in a container next to the services instead of in somebody's cloud console.

The first decision is how the services are named. Each gets an upstream block, so the address appears exactly once and the routes below refer to it by name. Adding a second replica of the catalogue is one more line inside the block and no change anywhere else.

```
• nginx.conf

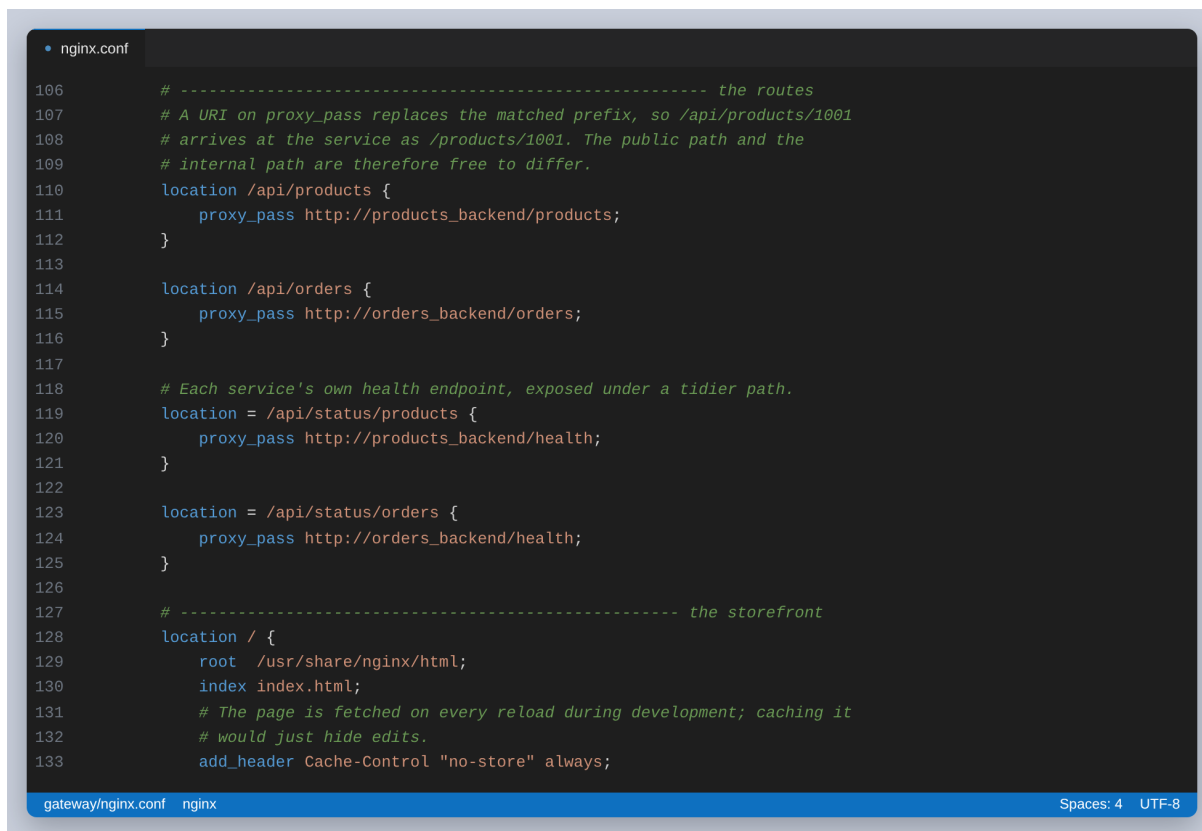
40  # One named upstream per microservice. Naming them rather than writing the
41  # address inline is what allows a second replica to be added on one line.
42  upstream products_backend {
43      server products-service:8000;
44      keepalive 16;
45  }
46
47  upstream orders_backend {
48      server orders-service:8001;
49      keepalive 16;
50  }
51
52  # HTTP/1.1 with an emptied Connection header is what lets the keepalive
53  # pools above actually be reused; on HTTP/1.0 every proxied request would
54  # open a new connection.
55  proxy_http_version 1.1;
56  proxy_set_header Connection "";
57
58  proxy_set_header Host          $host;
59  proxy_set_header X-Real-IP     $remote_addr;
60  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
61  proxy_set_header X-Forwarded-Proto $scheme;
62
63  # The correlation ID. NGINX generates $request_id per request; the services
64  # log it and hand it on, so one ID appears in all three container logs.
65  proxy_set_header X-Request-ID  $request_id;
66  proxy_set_header X-Gateway     nginx-api-gateway;
67
68  proxy_connect_timeout 2s;
69  proxy_send_timeout   5s;
70  proxy_read_timeout   5s;
71
72  # Off (the default, stated explicitly) so a 409 or 503 that a service
73  # generated on purpose reaches the client as written. Only errors NGINX
74  # itself produces get replaced by the handler below.
75  proxy_intercept_errors off;
76
77  # The services echo the correlation ID back, which is useful in their own
78  # logs and wrong on the way out: the gateway adds its own X-Request-ID, so
79  # the client received the header twice and the storefront displayed
80  # "<id>, <id>". Dropping the upstream copy leaves one, from the component
81  # that actually minted it.
82  proxy_hide_header X-Request-ID;
```

Figure 4: The two upstreams, and the proxy defaults that apply to both

The headers under them are the gateway's responsibility. X-Forwarded-For and X-Forwarded-Proto are there so a service can still tell who is asking after the hop. X-Request-ID is

the one that matters for Task 2: NGINX generates a fresh value per request and the services log it. The timeouts are deliberately short, so a service that has stopped answering becomes a fast error rather than a client that hangs.

The routes are prefixes. Because `proxy_pass` is given a URI as well as a host, the matched prefix is replaced rather than appended, so the public path and the internal path are free to differ.

A screenshot of a code editor showing the configuration file `nginx.conf`. The file is divided into two main sections: "the routes" and "the storefront". The "routes" section defines several `location` blocks for `/api/products`, `/api/orders`, `/api/status/products`, and `/api/status/orders`, each using `proxy_pass` to forward requests to specific backend services. The "storefront" section defines a `location /` block for the root path, setting the root directory and adding a `Cache-Control` header.

```
106 # ----- the routes
107 # A URI on proxy_pass replaces the matched prefix, so /api/products/1001
108 # arrives at the service as /products/1001. The public path and the
109 # internal path are therefore free to differ.
110 location /api/products {
111     proxy_pass http://products_backend/products;
112 }
113
114 location /api/orders {
115     proxy_pass http://orders_backend/orders;
116 }
117
118 # Each service's own health endpoint, exposed under a tidier path.
119 location = /api/status/products {
120     proxy_pass http://products_backend/health;
121 }
122
123 location = /api/status/orders {
124     proxy_pass http://orders_backend/health;
125 }
126
127 # ----- the storefront
128 location / {
129     root /usr/share/nginx/html;
130     index index.html;
131     # The page is fetched on every reload during development; caching it
132     # would just hide edits.
133     add_header Cache-Control "no-store" always;
```

Figure 5: Path-based routing. `/api/products/1002` reaches the service as `/products/1002`

Public route	Goes to	Methods
<code>/api/products</code>	products-service:8000/products	GET
<code>/api/products/{id}</code>	products-service:8000/products/{id}	GET
<code>/api/orders</code>	orders-service:8001/orders	GET, POST
<code>/api/orders/{id}</code>	orders-service:8001/orders/{id}	GET
<code>/api/orders/dependency</code>	orders-service:8001/orders/dependency	GET
<code>/api/status/products</code>	products-service:8000/health	GET
<code>/api/status/orders</code>	orders-service:8001/health	GET
<code>/health</code>	answered by the gateway itself	GET

Public route	Goes to	Methods
/	the storefront, served from disk	GET

The last two rows are why the storefront is worth having. The page and the API sit on one origin, so the JavaScript fetches `/api/products` as a same-origin request and there is no CORS configuration anywhere in this project. That is a real benefit of a gateway and it is easy to miss, because what it produces is the absence of work.

Finally, a gateway that returns NGINX's default HTML error page is a strange thing for a JSON API to do, so failures NGINX generates itself are shaped into JSON:

```

136 # ----- shaped failures
137 # Without this a stopped container gives the client NGINX's default HTML
138 # 502 page, which is a strange thing for a JSON API to return.
139 error_page 502 504 = @gateway_error;
140
141 location @gateway_error {
142     default_type application/json;
143     add_header X-Request-ID $request_id always;
144     add_header X-Served-By "api-gateway" always;
145     return 503 '{"error":"upstream service unavailable","gateway":"nginx-api-gateway","request_id":"$request_id"}';

```

Figure 6: A JSON 503 for failures NGINX produces itself

`proxy_intercept_errors` is left off, which is the default but worth stating, because it is what lets a 409 or a 503 that a service produced on purpose reach the client exactly as the service wrote it. Only errors NGINX generates get replaced.

Building it, and a check I had to remove

```
bash - docker build: three images

=== docker version ===
client 5.2.3

=== docker build ./products-service ===
=== docker build ./gateway ===
STEP 1/4: FROM nginx:1.27-alpine
STEP 2/4: COPY nginx.conf /etc/nginx/nginx.conf
--> 4f7cf3f523bb
STEP 3/4: COPY html/ /usr/share/nginx/html/
--> 490c804758ad
STEP 4/4: EXPOSE 8091
COMMIT bazaar/api-gateway:1.0
--> 8339fde0ac30
Successfully tagged localhost/bazaar/api-gateway:1.0
8339fde0ac3038306721073b52d877a99d107225f53457f9ffb10e98174ebab7
```

Figure 7: docker build, three images from three Dockerfiles

```
• Dockerfile
1 FROM nginx:1.27-alpine
2
3 # Replaces the stock configuration wholesale rather than dropping a file into
4 # conf.d, because the upstream blocks and the proxy defaults live in the http
5 # block and there is nothing in the default file worth keeping.
6 COPY nginx.conf /etc/nginx/nginx.conf
7
8 # The storefront. Serving it from the gateway is what puts the page and the API
9 # on one origin.
10 COPY html/ /usr/share/nginx/html/
11
12 # There is deliberately no 'RUN nginx -t' here, which was not the first plan.
13 # 'nginx -t' resolves every name in an upstream block, and during a build there
14 # is no network and no products-service to resolve, so the check fails on a
15 # configuration file that is perfectly valid:
16 #
17 # [emerg] host not found in upstream "products-service:8000"
18 #
19 # The same resolution happens when the container starts, which is why
20 # docker-compose.yml declares depends_on. The configuration is verified instead
21 # by run.sh, from inside the running container where the names resolve.
22
23 EXPOSE 8091

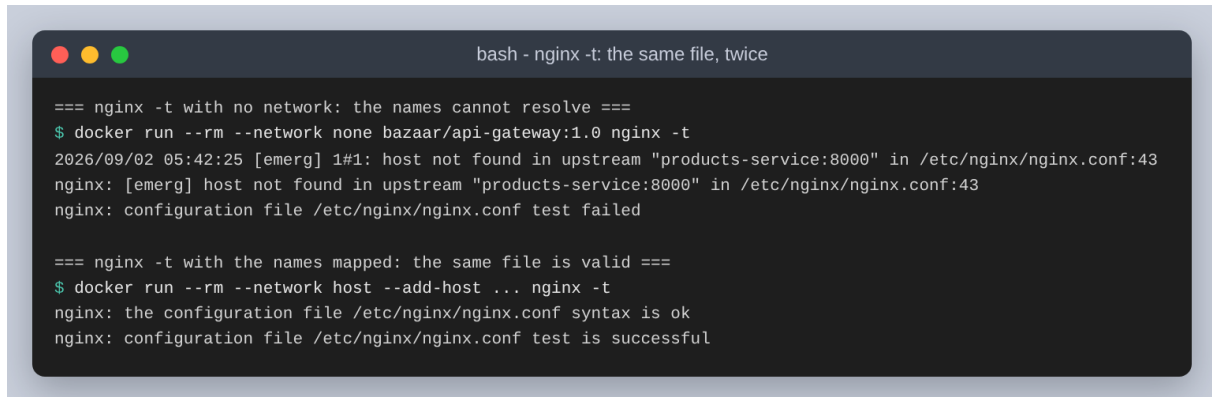
gateway/Dockerfile Dockerfile Spaces: 4 UTF-8
```

Figure 8: The gateway image, and the comment explaining what is missing from it

What is missing there is a validation step. I had written RUN nginx -t into the Dockerfile, on the reasoning that a bad directive should fail the build rather than produce an image that crash-loops. The build then failed on a file that was completely valid, because nginx -t resolves every name in

an upstream block, and during a build there is no network and no products-service to resolve.

Running the same check on the same image twice, with nothing different but whether the names resolve, shows exactly that:



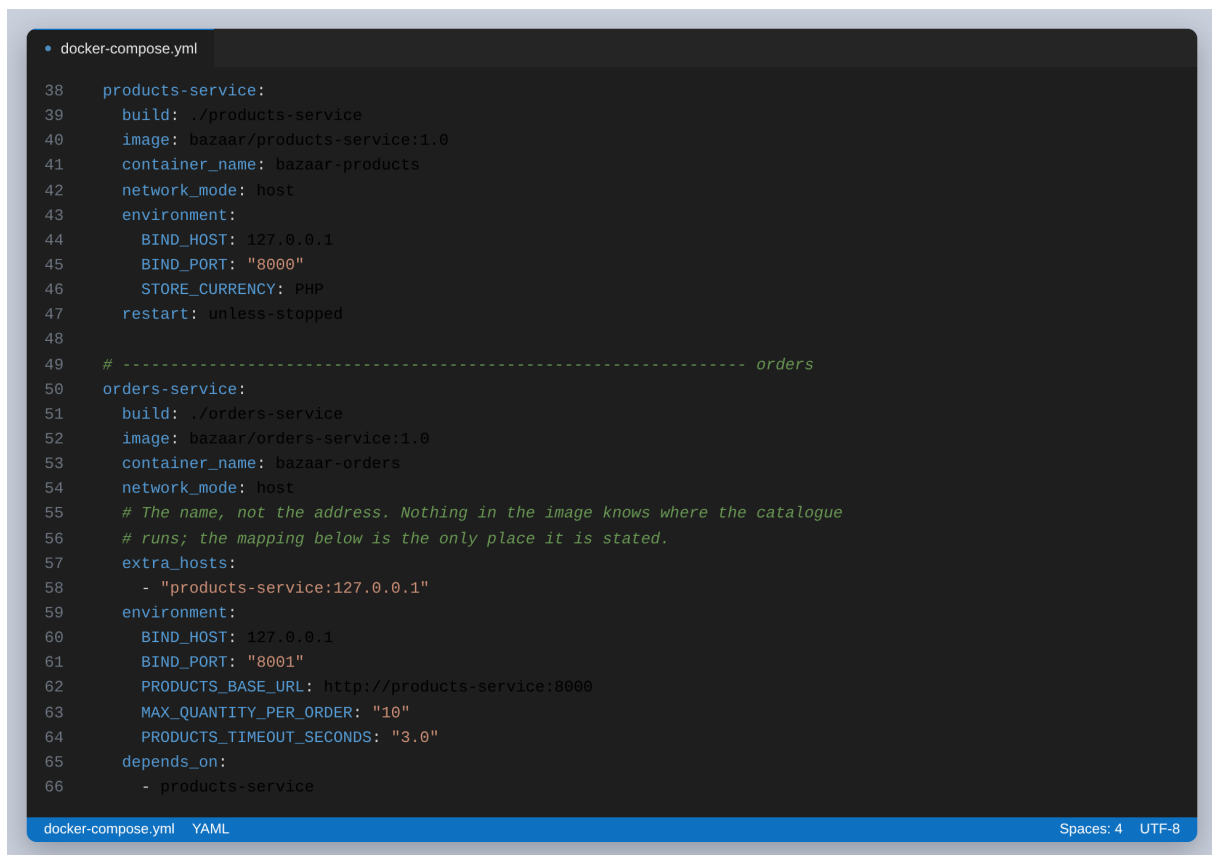
```
bash - nginx -t: the same file, twice

=== nginx -t with no network: the names cannot resolve ===
$ docker run --rm --network none bazaar/api-gateway:1.0 nginx -t
2026/09/02 05:42:25 [emerg] 1#1: host not found in upstream "products-service:8000" in /etc/nginx/nginx.conf:43
nginx: [emerg] host not found in upstream "products-service:8000" in /etc/nginx/nginx.conf:43
nginx: configuration file /etc/nginx/nginx.conf test failed

=== nginx -t with the names mapped: the same file is valid ===
$ docker run --rm --network host --add-host ... nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Figure 9: The same image and the same config file. Only name resolution differs

That is worth more than the build-time check I lost. NGINX resolves upstream names once, at startup, which is why the containers must exist before the gateway starts and why compose declares `depends_on`. It also comes back in the very last test.



```
• docker-compose.yml

38 products-service:
39   build: ./products-service
40   image: bazaar/products-service:1.0
41   container_name: bazaar-products
42   network_mode: host
43   environment:
44     BIND_HOST: 127.0.0.1
45     BIND_PORT: "8000"
46     STORE_CURRENCY: PHP
47   restart: unless-stopped
48
49 # ----- orders
50 orders-service:
51   build: ./orders-service
52   image: bazaar/orders-service:1.0
53   container_name: bazaar-orders
54   network_mode: host
55   # The name, not the address. Nothing in the image knows where the catalogue
56   # runs; the mapping below is the only place it is stated.
57   extra_hosts:
58     - "products-service:127.0.0.1"
59   environment:
60     BIND_HOST: 127.0.0.1
61     BIND_PORT: "8001"
62     PRODUCTS_BASE_URL: http://products-service:8000
63     MAX_QUANTITY_PER_ORDER: "10"
64     PRODUCTS_TIMEOUT_SECONDS: "3.0"
65   depends_on:
66     - products-service

docker-compose.yml  YAML  Spaces: 4  UTF-8
```

Figure 10: The two services in docker-compose.yml


```
• docker-compose.yml
70 gateway:
71   build: ./gateway
72   image: bazaar/api-gateway:1.0
73   container_name: bazaar-gateway
74   network_mode: host
75   # Both upstream names resolved here rather than written into nginx.conf.
76   extra_hosts:
77     - "products-service:127.0.0.1"
78     - "orders-service:127.0.0.1"
79   depends_on:
80     # Ordering only, not readiness: NGINX resolves every name in an upstream
81     # block when it starts, so the names have to exist by then.
82     - products-service
83     - orders-service
docker-compose.yml  YAML  Spaces: 4  UTF-8
```

Figure 11: The gateway. The names it needs are mapped here, not written into nginx.conf

With the images built, all three containers come up and the gateway is the only one with a reachable address:

```
bash - the three containers, running

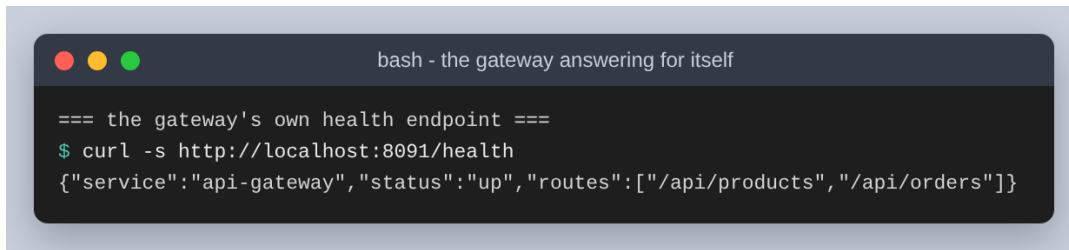
=== docker ps ===
$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Command}}'
NAMES          IMAGE                                STATUS        COMMAND
bazaar-products localhost/bazaar/products-service:1.0 Up 1 second   sh -c exec uvicor...
bazaar-orders  localhost/bazaar/orders-service:1.0 Up 1 second   sh -c exec uvicor...
bazaar-gateway localhost/bazaar/api-gateway:1.0     Up 1 second   nginx -g daemon o...

=== docker images ===
$ docker images bazaar/* --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}'
REPOSITORY      TAG      IMAGE ID      SIZE
localhost/bazaar/api-gateway  1.0      8339fde0ac30  49.7 MB
localhost/bazaar/orders-service 1.0      93e83c9e8918  173 MB
localhost/bazaar/products-service 1.0      f2411c61b8ba  171 MB

=== how each container is attached, and what it binds ===
CONTAINER      NETWORK  BINDS
bazaar-products host      127.0.0.1:8000 (loopback only)
bazaar-orders  host      127.0.0.1:8001 (loopback only)
bazaar-gateway host      0.0.0.0:8091 (reachable)
```

Figure 12: docker ps, the three images, and what each container binds

The gateway answers for itself without touching either service, which is what makes it useful to poll:



```
bash - the gateway answering for itself

=== the gateway's own health endpoint ===
$ curl -s http://localhost:8091/health
{"service":"api-gateway","status":"up","routes":["/api/products","/api/orders"]}
```

Figure 13: GET /health, answered by NGINX

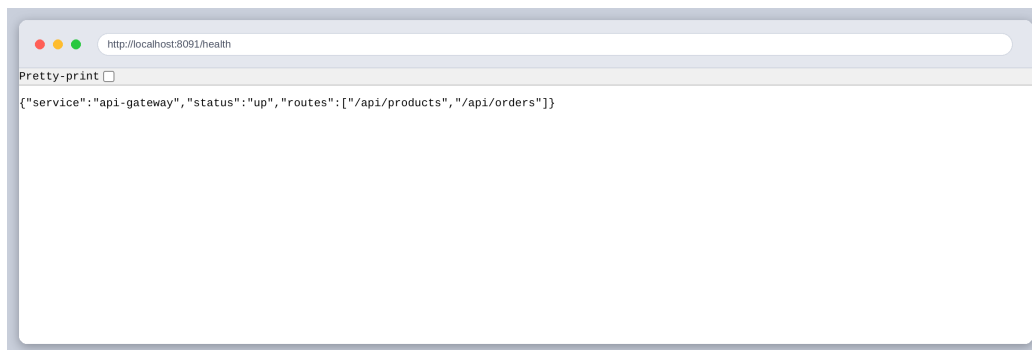
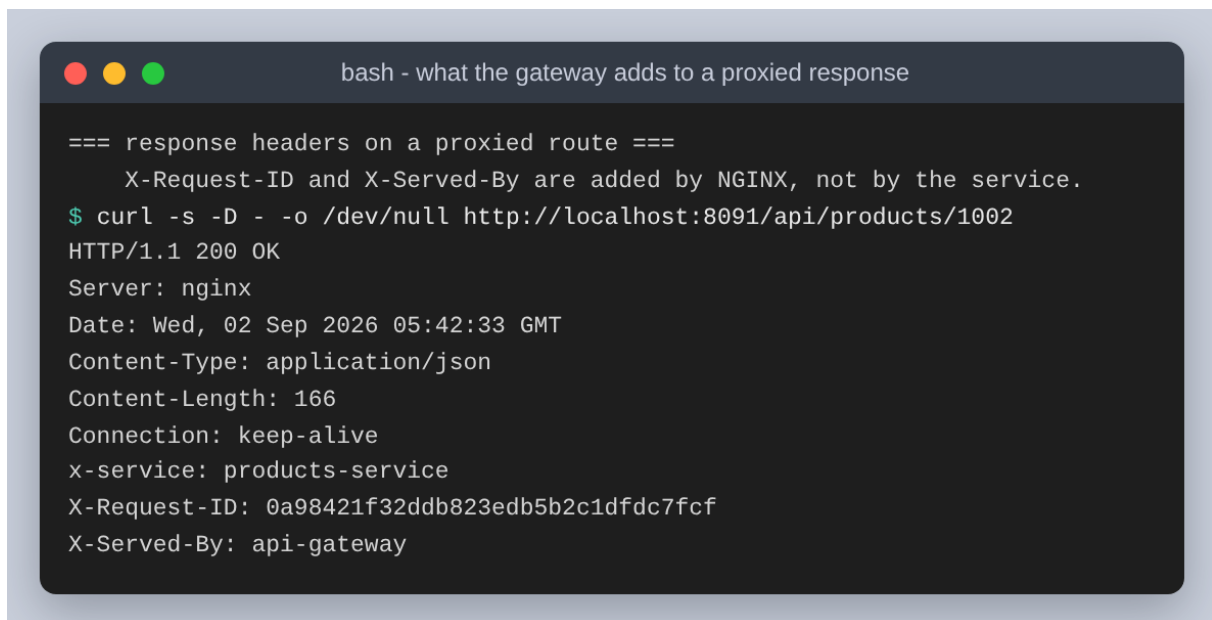


Figure 14: The same endpoint in a browser

And on a proxied route, the two headers the gateway adds come back with the response:



```
bash - what the gateway adds to a proxied response

=== response headers on a proxied route ===
X-Request-ID and X-Served-By are added by NGINX, not by the service.
$ curl -s -D - -o /dev/null http://localhost:8091/api/products/1002
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 02 Sep 2026 05:42:33 GMT
Content-Type: application/json
Content-Length: 166
Connection: keep-alive
x-service: products-service
X-Request-ID: 0a98421f32ddb823edb5b2c1dfdc7fcf
X-Served-By: api-gateway
```

Figure 15: X-Request-ID and X-Served-By are added by the gateway; x-service came from the service

Getting those headers onto every route took two attempts. NGINX does not merge `add_header` across levels: the moment a location block declares one of its own, every `add_header` inherited from the server block is dropped. My `/health` block set `Content-Type` that way, and `/health` came back with no `X-Request-ID` while every proxied route had one. The fix is to set the type with `default_type` and repeat the two headers inside the block, which is why they appear twice in the file.



```
bash - both services' health, reached through the gateway

=== each service's health, reached through the gateway ===
$ curl -s http://localhost:8091/api/status/products
{
  "service": "products-service",
  "status": "up",
  "instance": "ip-10-137-53-49.us-east-1.compute.internal",
  "products": 8
}
$ curl -s http://localhost:8091/api/status/orders
{
  "service": "orders-service",
  "status": "up",
  "instance": "ip-10-137-53-49.us-east-1.compute.internal",
  "orders": 0,
  "depends_on": "http://products-service:8000"
}
```

Figure 16: Both services' health, reached through the gateway under a tidier path than they use internally

Task 2: Service communication patterns

There are two directions of traffic here and they are not the same problem.

- North-south: a client to a service, through the gateway. The client knows one address and one set of paths. This is what Task 1 built.
- East-west: the orders service to the products service. No client is involved and no browser is waiting. This call does not go through the gateway at all.

I chose synchronous REST over HTTP for the east-west call rather than a message queue, and the reason is the nature of the work: the orders service cannot write an order until it knows the price. There is nothing to do while it waits, so a queue would add a broker to run and a reply to correlate without removing any waiting. A queue earns its place when the caller does not need the answer to continue, and pricing is the opposite of that.

The internal call also skips the gateway on purpose. Routing sideways traffic through the front door adds a hop to every internal request and makes the gateway a single point of failure for calls no client is waiting on.

```

• app.py
100 async def fetch_product(product_id: int, rid: str) -> dict[str, Any]:
101     """Ask the products service about one product.
102
103     Every failure mode is translated into a status code the caller can act on.
104     Letting an httpx exception escape would surface as a 500, which tells a
105     client nothing about whose fault it was.
106     """
107     url = f"{PRODUCTS_BASE_URL}/products/{product_id}"
108     log.info("rid=%s -> GET %s", rid, url)
109     try:
110         async with httpx.AsyncClient(timeout=TIMEOUT) as client:
111             reply = await client.get(
112                 url, headers={"X-Request-ID": rid, "X-Called-By": SERVICE})
113     except (httpx.ConnectError, httpx.ConnectTimeout) as exc:
114         log.warning("rid=%s products-service unreachable: %s", rid, exc)
115         raise HTTPException(
116             status_code=503,
117             detail={"error": "products service unavailable",
118                   "dependency": PRODUCTS_BASE_URL,
119                   "hint": "the catalogue container is not answering, so the "
120                         "order cannot be priced",
121                   "reason": type(exc).__name__}) from exc
122     except httpx.ReadTimeout as exc:
123         raise HTTPException(
124             status_code=504,
125             detail={"error": "products service timed out",
126                   "dependency": PRODUCTS_BASE_URL,
127                   "timeout_seconds": TIMEOUT}) from exc
128
129     if reply.status_code == 404:
130         raise HTTPException(
131             status_code=404,
132             detail=f"product {product_id} is not in the catalogue")
133     if reply.status_code >= 500:
134         raise HTTPException(
135             status_code=502,
136             detail={"error": "products service returned an error",
137                   "upstream_status": reply.status_code})
138     return reply.json()

```

orders-service/app.py Python

Spaces: 4 UTF-8

Figure 17: The east-west call. Every failure mode becomes a status code the caller can act on

Most of that function is error translation, and that is the point of it. If an httpx exception were allowed to escape, a stopped catalogue would surface to the customer as a 500, which says only that something broke. Instead a refused connection becomes a 503 naming the dependency, a slow answer becomes a 504 with the timeout that was exceeded, and the catalogue's own 404 is passed along as a 404.

```

• app.py
196 @app.post("/orders", status_code=201, tags=["orders"])
197 async def place_order(body: OrderRequest, request: Request) -> dict[str, Any]:
198     """Create an order, pricing it through the products service."""
199     rid = request.headers.get("x-request-id", "-")
200
201     if body.quantity > MAX_QUANTITY:
202         raise HTTPException(
203             status_code=422,
204             detail={"error": "quantity above the configured maximum",
205                   "requested": body.quantity, "maximum": MAX_QUANTITY})
206
207     product = await fetch_product(body.product_id, rid)
208
209     if product["stock"] == 0:
210         raise HTTPException(
211             status_code=409,
212             detail={"error": "out of stock", "product": product["name"],
213                   "product_id": product["id"]})
214     if body.quantity > product["stock"]:
215         raise HTTPException(
216             status_code=409,
217             detail={"error": "not enough stock", "product": product["name"],
218                   "requested": body.quantity, "available": product["stock"]})
219
220     order_id = _next_id[0]
221     _next_id[0] += 1
222     unit = float(product["price"])
223     order = {
224         "id": order_id,
225         "customer": body.customer,
226         "product_id": product["id"],
227         "product_name": product["name"],
228         "unit_price": unit,
229         "quantity": body.quantity,
230         "total": round(unit * body.quantity, 2),
231         "currency": product.get("currency", "PHP"),
232         "status": "CONFIRMED",
233         "placed_at": datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%SZ"),
234         # Recorded on the order itself, so the trail survives the log buffer.
235         "priced_by": product.get("served_by", "products-service"),
236         "request_id": rid,
237     }
238     ORDERS[order_id] = order
239     log.info("rid=%s order %s confirmed: %s x%s = %.2f",
240             rid, order_id, product["name"], body.quantity, order["total"])
241     return order

```

orders-service/app.py Python

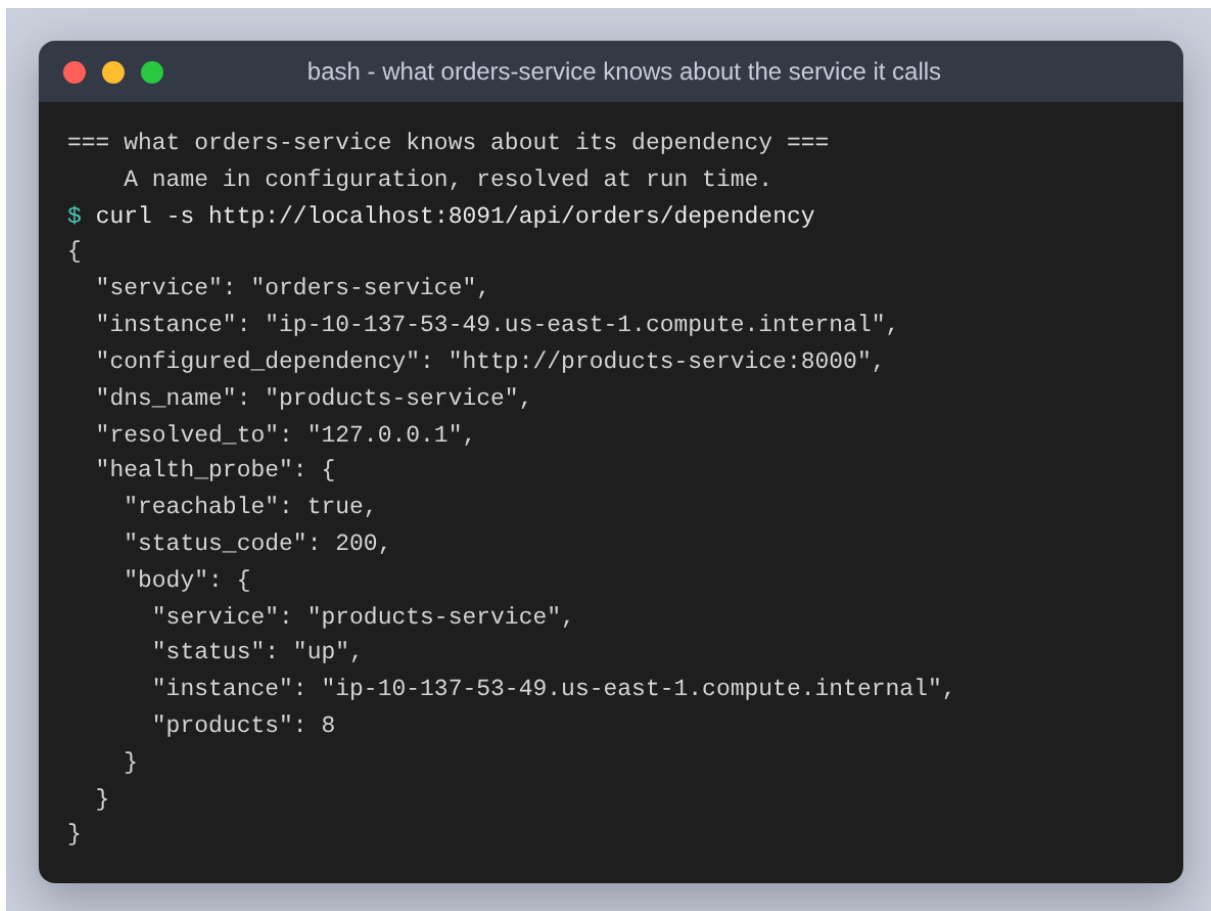
Spaces: 4 UTF-8

Figure 18: Placing an order: the local rule first, then the call, then the stock checks

The quantity limit is checked before the call, because there is no reason to ask the catalogue about an order that is already invalid. The stock checks have to come after it, because stock is the catalogue's data and the orders service has no copy of it.

Where the address comes from

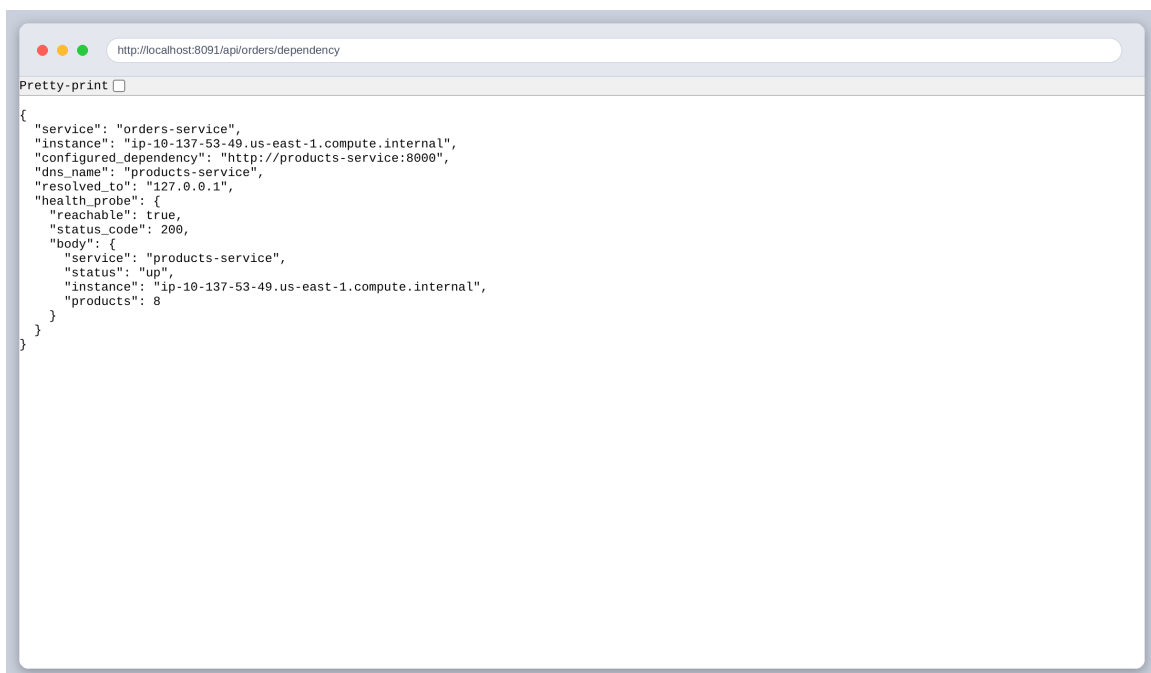
Nothing in the orders service image knows where the catalogue runs. It reads a name from `PRODUCTS_BASE_URL` and the name is resolved when the call is made. The service will report what it currently sees:



```
bash - what orders-service knows about the service it calls

=== what orders-service knows about its dependency ===
    A name in configuration, resolved at run time.
$ curl -s http://localhost:8091/api/orders/dependency
{
  "service": "orders-service",
  "instance": "ip-10-137-53-49.us-east-1.compute.internal",
  "configured_dependency": "http://products-service:8000",
  "dns_name": "products-service",
  "resolved_to": "127.0.0.1",
  "health_probe": {
    "reachable": true,
    "status_code": 200,
    "body": {
      "service": "products-service",
      "status": "up",
      "instance": "ip-10-137-53-49.us-east-1.compute.internal",
      "products": 8
    }
  }
}
```

Figure 19: GET /api/orders/dependency, through the gateway



```
http://localhost:8091/api/orders/dependency

Pretty-print ☐

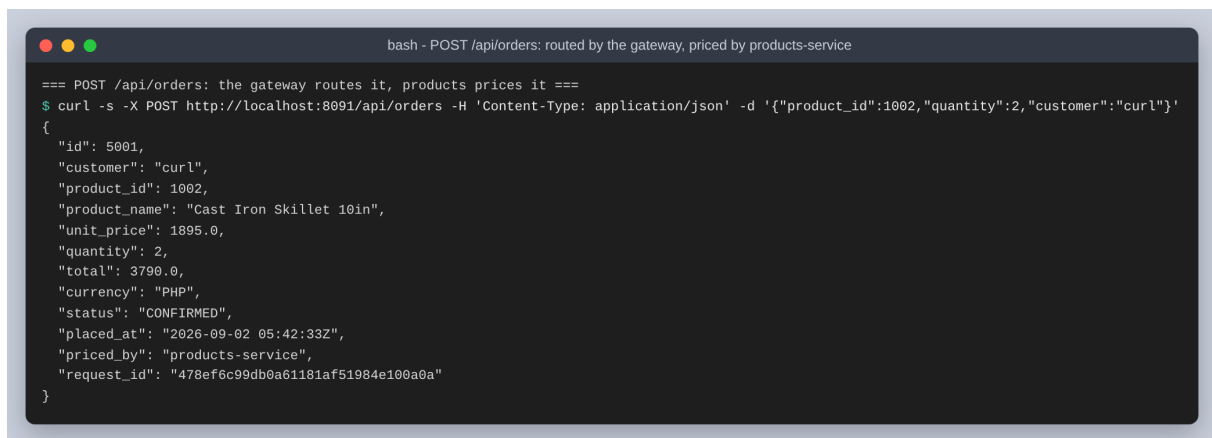
{
  "service": "orders-service",
  "instance": "ip-10-137-53-49.us-east-1.compute.internal",
  "configured_dependency": "http://products-service:8000",
  "dns_name": "products-service",
  "resolved_to": "127.0.0.1",
  "health_probe": {
    "reachable": true,
    "status_code": 200,
    "body": {
      "service": "products-service",
      "status": "up",
      "instance": "ip-10-137-53-49.us-east-1.compute.internal",
      "products": 8
    }
  }
}
```

Figure 20: The same view in a browser

The configured value is a name, the address underneath it was resolved at run time, and the health probe was made just now. On the bridge network this was meant to run on, that address would have been the catalogue container's address, resolved by the runtime's DNS. Here it resolves to loopback. The mechanism is the same either way: the code holds a name and the environment supplies the mapping.

An order, end to end

A POST to `/api/orders` goes through both patterns in one request. The gateway routes it north-south to the orders service; the orders service calls the catalogue east-west to price it; the order comes back with a total the orders service never had the data to compute on its own.



```
bash - POST /api/orders: routed by the gateway, priced by products-service

=== POST /api/orders: the gateway routes it, products prices it ===
$ curl -s -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' -d '{"product_id":1002,"quantity":2,"customer":"curl"}'
{
  "id": 5001,
  "customer": "curl",
  "product_id": 1002,
  "product_name": "Cast Iron Skillet 10in",
  "unit_price": 1895.0,
  "quantity": 2,
  "total": 3790.0,
  "currency": "PHP",
  "status": "CONFIRMED",
  "placed_at": "2026-09-02 05:42:33Z",
  "priced_by": "products-service",
  "request_id": "478ef6c99db0a61181af51984e100a0a"
}
```

Figure 21: One POST `/api/orders`. `unit_price` and `priced_by` both came from the other service

Following one request through three containers

This is the part I would keep if I had to throw the rest away. The gateway stamps every incoming request with an X-Request-ID. The orders service logs it and forwards it on its own outbound call. The products service logs it again, with who called. So one customer request leaves the same identifier in three separate container logs, and grep is enough to reconstruct the whole path.


```
bash - one request id, three container logs

=== one POST /api/orders, and the same ID in three container logs ===
The gateway generated it. Nobody configured it anywhere.

$ RID=f5dd287e3a84d4449cafff9815f30f682

$ docker logs bazaar-gateway | grep $RID
127.0.0.1 "POST /api/orders HTTP/1.1" 201 334b 0.014s rid=f5dd287e3a84d4449cafff9815f30f682 upstream=127.0.0.1:8001

$ docker logs bazaar-orders | grep $RID
05:42:33 orders-service rid=f5dd287e3a84d4449cafff9815f30f682 -> GET http://products-service:8000/products/1001
05:42:33 orders-service rid=f5dd287e3a84d4449cafff9815f30f682 order 5003 confirmed: Bamboo Cutting Board x4 = 2996.00
05:42:33 orders-service rid=f5dd287e3a84d4449cafff9815f30f682 POST /orders -> 201

$ docker logs bazaar-products | grep $RID
05:42:33 products-service rid=f5dd287e3a84d4449cafff9815f30f682 from=orders-service GET /products/1001 -> 200
```

Figure 22: One POST /api/orders. The id f5dd287e3a84 appears in all three logs

Read top to bottom that is the entire journey of one request: NGINX accepted it and sent it to 127.0.0.1:8001, the orders service asked products-service for a price on product 1001, the catalogue answered 200 and recorded that the caller was orders-service, and the order was confirmed. Nobody configured the identifier. NGINX generated it, and the only reason it travels is one proxy_set_header line plus the header the orders service copies onto its outbound call.

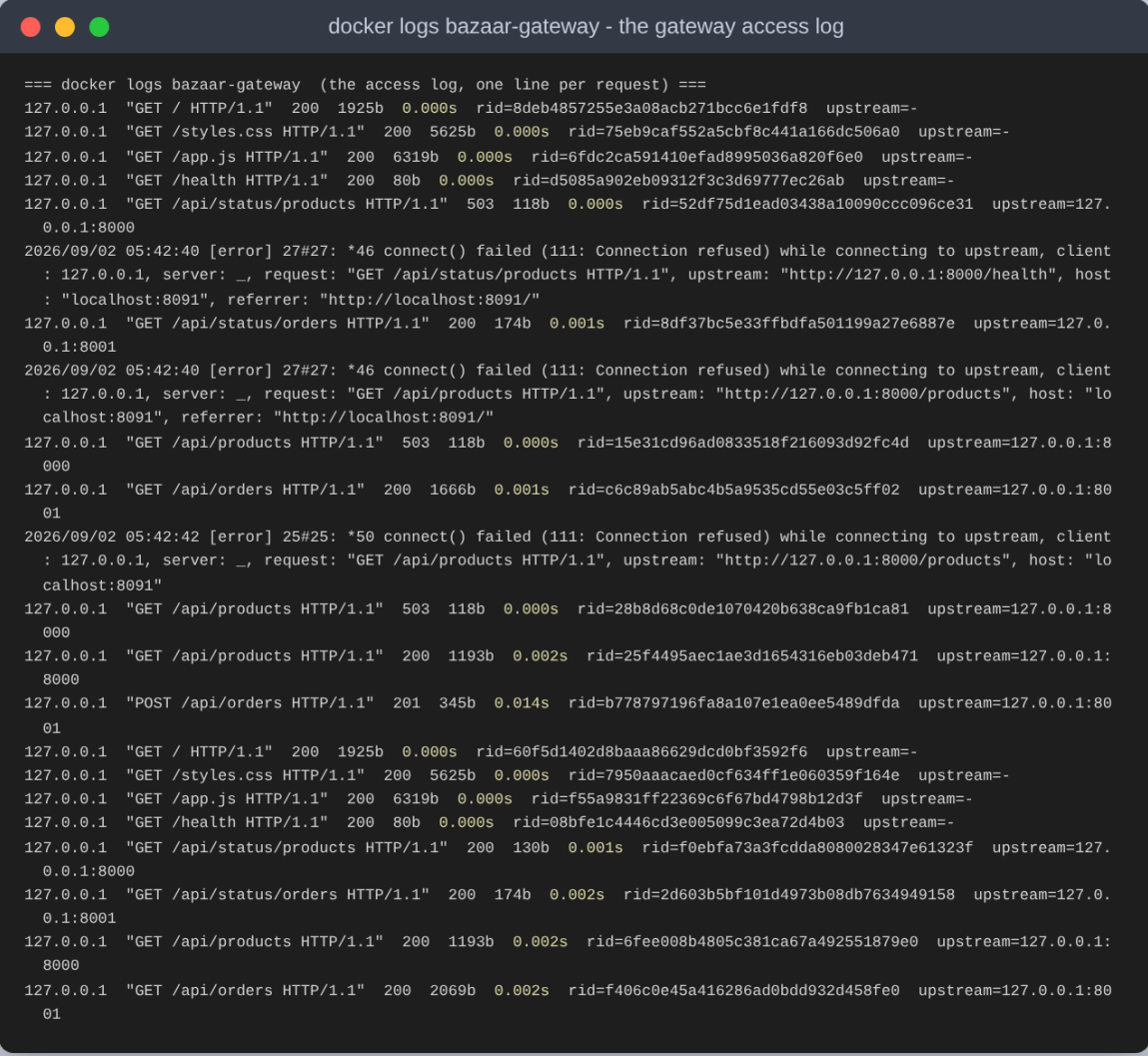
It is also stored on the order, so a support question about one order can be turned back into the log lines that produced it:

```
bash - the same id, recorded on the order

=== and it is stored on the order itself ===
$ curl -s http://localhost:8091/api/orders/5003
{
  "id": 5003,
  "customer": "trace",
  "product_id": 1001,
  "product_name": "Bamboo Cutting Board",
  "unit_price": 749.0,
  "quantity": 4,
  "total": 2996.0,
  "currency": "PHP",
  "status": "CONFIRMED",
  "placed_at": "2026-09-02 05:42:33Z",
  "priced_by": "products-service",
  "request_id": "f5dd287e3a84d4449cafff9815f30f682"
}
```

Figure 23: The same id, recorded on the order

The gateway's access log is worth a look on its own, because \$upstream_addr records which container actually served each request. That is the difference between believing the routing works and being able to show it.



```
=== docker logs bazaar-gateway (the access log, one line per request) ===
127.0.0.1 "GET / HTTP/1.1" 200 1925b 0.000s rid=8deb4857255e3a08acb271bcc6e1fdf8 upstream=-
127.0.0.1 "GET /styles.css HTTP/1.1" 200 5625b 0.000s rid=75eb9caf552a5cbf8c441a166dc506a0 upstream=-
127.0.0.1 "GET /app.js HTTP/1.1" 200 6319b 0.000s rid=6fdc2ca591410efad8995036a820f6e0 upstream=-
127.0.0.1 "GET /health HTTP/1.1" 200 80b 0.000s rid=d5085a902eb09312f3c3d69777ec26ab upstream=-
127.0.0.1 "GET /api/status/products HTTP/1.1" 503 118b 0.000s rid=52df75d1ead03438a10090ccc096ce31 upstream=127.0.0.1:8000
2026/09/02 05:42:40 [error] 27#27: *46 connect() failed (111: Connection refused) while connecting to upstream, client
: 127.0.0.1, server: _, request: "GET /api/status/products HTTP/1.1", upstream: "http://127.0.0.1:8000/health", host
: "localhost:8091", referer: "http://localhost:8091/"
127.0.0.1 "GET /api/status/orders HTTP/1.1" 200 174b 0.001s rid=8df37bc5e33ffbd5a501199a27e6887e upstream=127.0.0.1:8001
2026/09/02 05:42:40 [error] 27#27: *46 connect() failed (111: Connection refused) while connecting to upstream, client
: 127.0.0.1, server: _, request: "GET /api/products HTTP/1.1", upstream: "http://127.0.0.1:8000/products", host: "lo
calhost:8091", referer: "http://localhost:8091/"
127.0.0.1 "GET /api/products HTTP/1.1" 503 118b 0.000s rid=15e31cd96ad0833518f216093d92fc4d upstream=127.0.0.1:8000
127.0.0.1 "GET /api/orders HTTP/1.1" 200 1666b 0.001s rid=c6c89ab5abc4b5a9535cd55e03c5ff02 upstream=127.0.0.1:8001
2026/09/02 05:42:42 [error] 25#25: *50 connect() failed (111: Connection refused) while connecting to upstream, client
: 127.0.0.1, server: _, request: "GET /api/products HTTP/1.1", upstream: "http://127.0.0.1:8000/products", host: "lo
calhost:8091"
127.0.0.1 "GET /api/products HTTP/1.1" 503 118b 0.000s rid=28b8d68c0de1070420b638ca9fb1ca81 upstream=127.0.0.1:8000
127.0.0.1 "GET /api/products HTTP/1.1" 200 1193b 0.002s rid=25f4495aeca1ae3d1654316eb03deb471 upstream=127.0.0.1:8000
127.0.0.1 "POST /api/orders HTTP/1.1" 201 345b 0.014s rid=b778797196fa8a107e1ea0ee5489dfda upstream=127.0.0.1:8001
127.0.0.1 "GET / HTTP/1.1" 200 1925b 0.000s rid=60f5d1402d8baaa86629dcd0bf3592f6 upstream=-
127.0.0.1 "GET /styles.css HTTP/1.1" 200 5625b 0.000s rid=7950aaacaed0cf634ff1e060359f164e upstream=-
127.0.0.1 "GET /app.js HTTP/1.1" 200 6319b 0.000s rid=f55a9831ff22369c6f67bd4798b12d3f upstream=-
127.0.0.1 "GET /health HTTP/1.1" 200 80b 0.000s rid=08bfe1c4446cd3e005099c3ea72d4b03 upstream=-
127.0.0.1 "GET /api/status/products HTTP/1.1" 200 130b 0.001s rid=f0ebfa73a3fcdda8080029347e61323f upstream=127.0.0.1:8000
127.0.0.1 "GET /api/status/orders HTTP/1.1" 200 174b 0.002s rid=2d603b5bf101d4973b08db7634949158 upstream=127.0.0.1:8001
127.0.0.1 "GET /api/products HTTP/1.1" 200 1193b 0.002s rid=6fee008b4805c381ca67a492551879e0 upstream=127.0.0.1:8000
127.0.0.1 "GET /api/orders HTTP/1.1" 200 2069b 0.002s rid=f406c0e45a416286ad0bdd932d458fe0 upstream=127.0.0.1:8001
```

Figure 24: The gateway access log. The upstream column shows :8000 or :8001 per request

```
docker logs bazaar-orders - every outbound call, with its id

=== docker logs bazaar-orders ===
05:42:34 orders-service HTTP Request: GET http://products-service:8000/products/4242 "HTTP/1.1 404 Not Found"
05:42:34 orders-service rid=2e6d353d912e4327fff09574ed68f0ec POST /orders -> 404
05:42:34 orders-service rid=6123708a8ba2a38f67de8ab86e982a98 -> GET http://products-service:8000/products/1005
05:42:34 orders-service HTTP Request: GET http://products-service:8000/products/1005 "HTTP/1.1 200 OK"
05:42:34 orders-service rid=6123708a8ba2a38f67de8ab86e982a98 order 5004 confirmed: Abaca Table Runner x2 = 790.00
05:42:34 orders-service rid=6123708a8ba2a38f67de8ab86e982a98 POST /orders -> 201
05:42:35 orders-service rid=ab3ac8abdae29d54764076484e52ea2b GET /health -> 200
05:42:35 orders-service rid=985646e2f3d14d9a085403d3902fcc7a GET /orders -> 200
05:42:37 orders-service rid=83189c9c9047978907c4be7e5c150811 GET /orders -> 200
05:42:38 orders-service HTTP Request: GET http://products-service:8000/health "HTTP/1.1 200 OK"
05:42:38 orders-service rid=6b0cdf1d1d0efaf3bcbcd4d1f4ba9ca79 GET /orders/dependency -> 200
05:42:40 orders-service rid=b068a23109b8d9bb27263722bb09d9b6 -> GET http://products-service:8000/products/1002
05:42:40 orders-service rid=b068a23109b8d9bb27263722bb09d9b6 products-service unreachable: All connection attempts failed
05:42:40 orders-service rid=b068a23109b8d9bb27263722bb09d9b6 POST /orders -> 503
05:42:40 orders-service rid=d5245541862ee6e74a5ef77de5d4a964 GET /orders -> 200
05:42:40 orders-service rid=94a00efb928594e7b103e35b8c3a2cbb GET /orders/dependency -> 200
05:42:40 orders-service rid=8df37bc5e33ffbfdfa501199a27e6887e GET /health -> 200
05:42:40 orders-service rid=c6c89ab5abc4b5a9535cd55e03c5ff02 GET /orders -> 200
05:42:44 orders-service rid=b778797196fa8a107e1ea0ee5489dfda -> GET http://products-service:8000/products/1007
05:42:44 orders-service HTTP Request: GET http://products-service:8000/products/1007 "HTTP/1.1 200 OK"
05:42:44 orders-service rid=b778797196fa8a107e1ea0ee5489dfda order 5005 confirmed: Calamansi Marmalade 250g x2 = 370.00
05:42:44 orders-service rid=b778797196fa8a107e1ea0ee5489dfda POST /orders -> 201
05:42:45 orders-service rid=2d603b5bf101d4973b08db7634949158 GET /health -> 200
05:42:45 orders-service rid=f406c0e45a416286ad0bdd932d458fe0 GET /orders -> 200
```

Figure 25: The orders service log. Every outbound call is logged before it is made

```
docker logs bazaar-products - who called, and what for

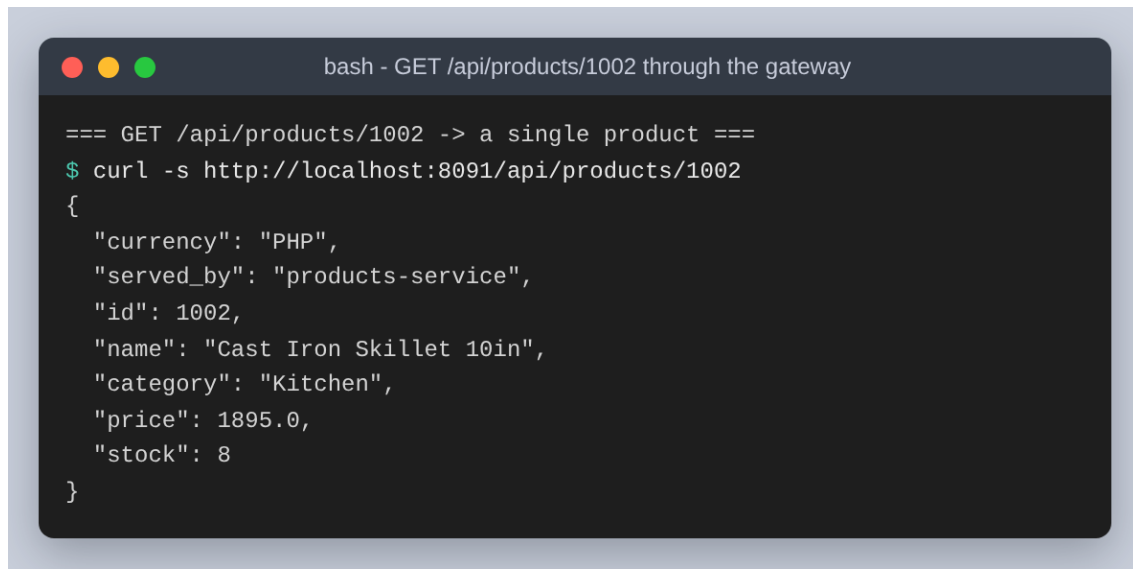
=== docker logs bazaar-products ===
05:42:33 products-service rid=f5dd287e3a84d4449caff9815f30f682 from=orders-service GET /products/1001 -> 200
05:42:34 products-service rid=bf34baf8d960e729a163d713463d5e99 from=orders-service GET /products/1006 -> 200
05:42:34 products-service rid=3f17373053636b11c6fc2ef2eb446344 from=orders-service GET /products/1004 -> 200
05:42:34 products-service rid=2e6d353d912e4327fff09574ed68f0ec from=orders-service GET /products/4242 -> 404
05:42:34 products-service rid=6123708a8ba2a38f67de8ab86e982a98 from=orders-service GET /products/1005 -> 200
05:42:35 products-service rid=efc497864a7e8df0c4a6ec9aeb0c2e37 from=gateway GET /health -> 200
05:42:35 products-service rid=9347190343a6d8b2a5c8866955a1e877 from=gateway GET /products -> 200
05:42:36 products-service rid=48fdad8d6876a97b87090757220168d8 from=gateway GET /products -> 200
05:42:38 products-service rid=- from=gateway GET /health -> 200
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
05:42:44 products-service rid=- from=gateway GET /health -> 200
05:42:44 products-service rid=25f4495aec1ae3d1654316eb03deb471 from=gateway GET /products -> 200
05:42:44 products-service rid=b778797196fa8a107e1ea0ee5489dfda from=orders-service GET /products/1007 -> 200
05:42:45 products-service rid=f0ebfa73a3fcdda8080028347e61323f from=gateway GET /health -> 200
05:42:45 products-service rid=6fee008b4805c381ca67a492551879e0 from=gateway GET /products -> 200
```

Figure 26: The catalogue log. from= distinguishes the gateway from the orders service

Task 3: Testing the gateway and the service communication

I tested with curl for the API and a real browser for the storefront. All of it goes through `http://localhost:8091`, and no test addresses a service directly apart from one comparison where that is the point.

Routing

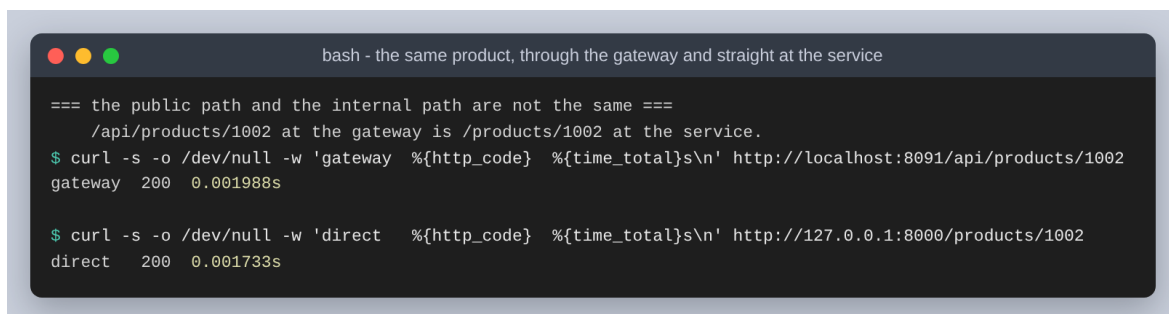


```
bash - GET /api/products/1002 through the gateway

=== GET /api/products/1002 -> a single product ===
$ curl -s http://localhost:8091/api/products/1002
{
  "currency": "PHP",
  "served_by": "products-service",
  "id": 1002,
  "name": "Cast Iron Skillet 10in",
  "category": "Kitchen",
  "price": 1895.0,
  "stock": 8
}
```

Figure 27: GET /api/products/1002. `served_by` names the service that answered

The same product fetched twice, once through the gateway and once straight at the service on loopback: 2.0 ms through the gateway against 1.7 ms direct.



```
bash - the same product, through the gateway and straight at the service

=== the public path and the internal path are not the same ===
/api/products/1002 at the gateway is /products/1002 at the service.
$ curl -s -o /dev/null -w 'gateway %{http_code} %{time_total}s\n' http://localhost:8091/api/products/1002
gateway 200 0.001988s

$ curl -s -o /dev/null -w 'direct %{http_code} %{time_total}s\n' http://127.0.0.1:8000/products/1002
direct 200 0.001733s
```

Figure 28: The extra hop, measured

The gateway came out marginally faster, which is not a real result. Both numbers are around a millisecond and a millisecond is mostly noise on loopback; run it again and they swap. What it does establish is that the extra hop costs nothing measurable at this scale, which is the useful half of the question.

```
bash - a query string, passed through untouched

=== a filter, passed through as a query string ===
$ curl -s 'http://localhost:8091/api/products?category=Pantry'
{
  "currency": "PHP",
  "count": 2,
  "served_by": "products-service",
  "products": [
    {
      "id": 1003,
      "name": "Barako Coffee Beans 1kg",
      "category": "Pantry",
      "price": 620.0,
      "stock": 52
    },
    {
      "id": 1007,
      "name": "Calamansi Marmalade 250g",
      "category": "Pantry",
      "price": 185.0,
      "stock": 41
    }
  ]
}
```

Figure 29: A query string, passed through untouched, so filtering stays the service's job

```
bash - a 404 from the service, not from the gateway

=== GET /api/products/9999 -> the service's own 404, not the gateway's ===
$ curl -s -w '[HTTP %{http_code}]\n' http://localhost:8091/api/products/9999
{
  "detail": "no product with id 9999 in the catalogue"
}[HTTP 404]
```

Figure 30: An id that is not in the catalogue gives the service's own 404, not a generic gateway error

The refusals

Four requests that should not succeed. The interesting thing is not that they fail but that each fails at the layer that owns the rule, with a status code that says which:

Request	Status	Refused by	Why
1006, qty 1	409	orders-service	Stock is zero. Needs the catalogue's answer first.
1004, qty 5	409	orders-service	Only three in stock. Also needs the catalogue.
1003, qty 25	422	orders-service	Over the configured maximum of 10. No call is made.
4242, qty 1	404	products-service	No such product. The 404 travels back through two hops.

```

bash - two refusals that need the catalogue to answer first

=== a product that is out of stock (1006, stock 0) ===
$ curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' \
  -d '{"product_id":1006,"quantity":1}'
{
  "detail": {
    "error": "out of stock",
    "product": "Stainless Tumbler 500ml",
    "product_id": 1006
  }
}
[HTTP 409]

=== more than the catalogue has (1004, stock 3) ===
$ curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' \
  -d '{"product_id":1004,"quantity":5}'
{
  "detail": {
    "error": "not enough stock",
    "product": "Buri Storage Basket",
    "requested": 5,
    "available": 3
  }
}
[HTTP 409]

```

Figure 31: Two refusals that both required the catalogue to answer first

```
bash - a local rule, and a 404 that travelled back from products-service

=== above the configured per-order maximum of 10 ===
Refused by orders-service before it calls products-service at all.
$ curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' -d '{"product_id":1003,"quantity":25}'
{
  "detail": {
    "error": "quantity above the configured maximum",
    "requested": 25,
    "maximum": 10
  }
}[HTTP 422]

=== a product that does not exist: the 404 comes from products-service ===
$ curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' -d '{"product_id":4242,"quantity":1}'
{
  "detail": "product 4242 is not in the catalogue"
}[HTTP 404]
```

Figure 32: A purely local rule, and a 404 that travelled back from the other service

The 422 shows the boundary clearly. The quantity limit is the orders service's own configuration, so it refuses without ever contacting the catalogue, and the products service log has no line for that request.

The storefront

```
• app.js

14 /* One wrapper around fetch, so every call updates the trace bar. The headers it
15 * reads are added by NGINX, not by the services. */
16 async function call(path, options = {}) {
17   const started = performance.now();
18   const response = await fetch(path, options);
19   const ms = Math.round(performance.now() - started);
20
21   const method = (options.method || "GET").toUpperCase();
22   set("t-request", `${method} ${path}`);
23   set("t-status", `${response.status} ${response.statusText}`);
24   set("t-served", response.headers.get("x-served-by") || "\u2014");
25   set("t-rid", response.headers.get("x-request-id") || "\u2014");
26   set("t-time", `${ms} ms`);
27
28   let body = null;
29   try { body = await response.json(); } catch { /* empty or not JSON */ }
30   return { ok: response.ok, status: response.status, body };
}
```

Figure 33: The only place the storefront talks to the API. Every path is relative

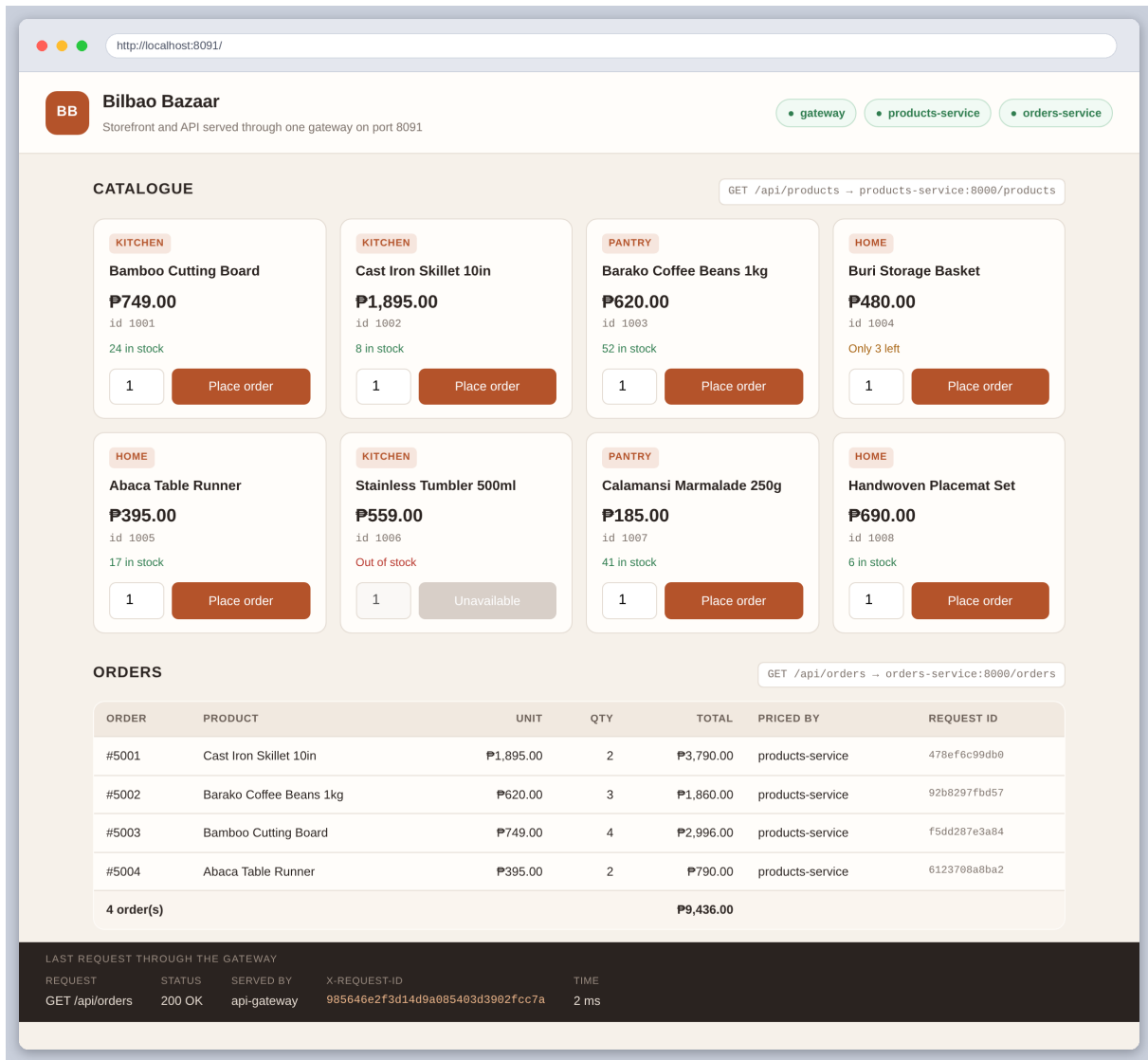


Figure 34: The storefront in Chromium. Eight products from products-service, orders from orders-service, and the gateway's own headers along the bottom

Everything on that page arrived through the gateway. The three pills top right are live polls of `/health`, `/api/status/products` and `/api/status/orders`. The catalogue is a `GET /api/products`. Each Place order button is a `POST /api/orders`, so every row in the orders table is a service-to-service call that has already happened: the priced by column names products-service, and the 5650.0 peso total was computed from prices the orders service had to ask for.

Product 1006 is the one to look at. It shows Out of stock with its button disabled, and the storefront was not told which product that is. It renders whatever stock the catalogue reports.

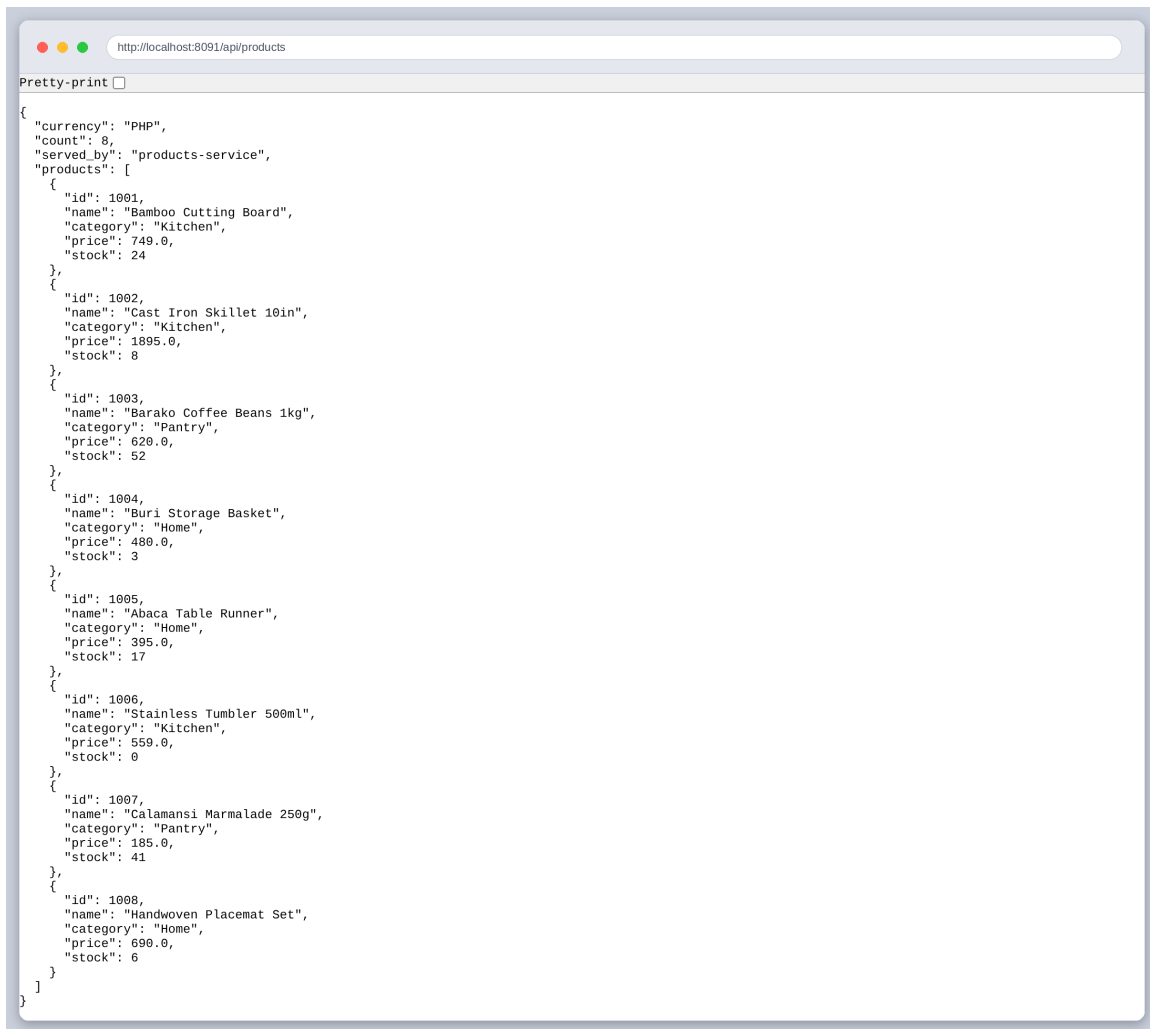
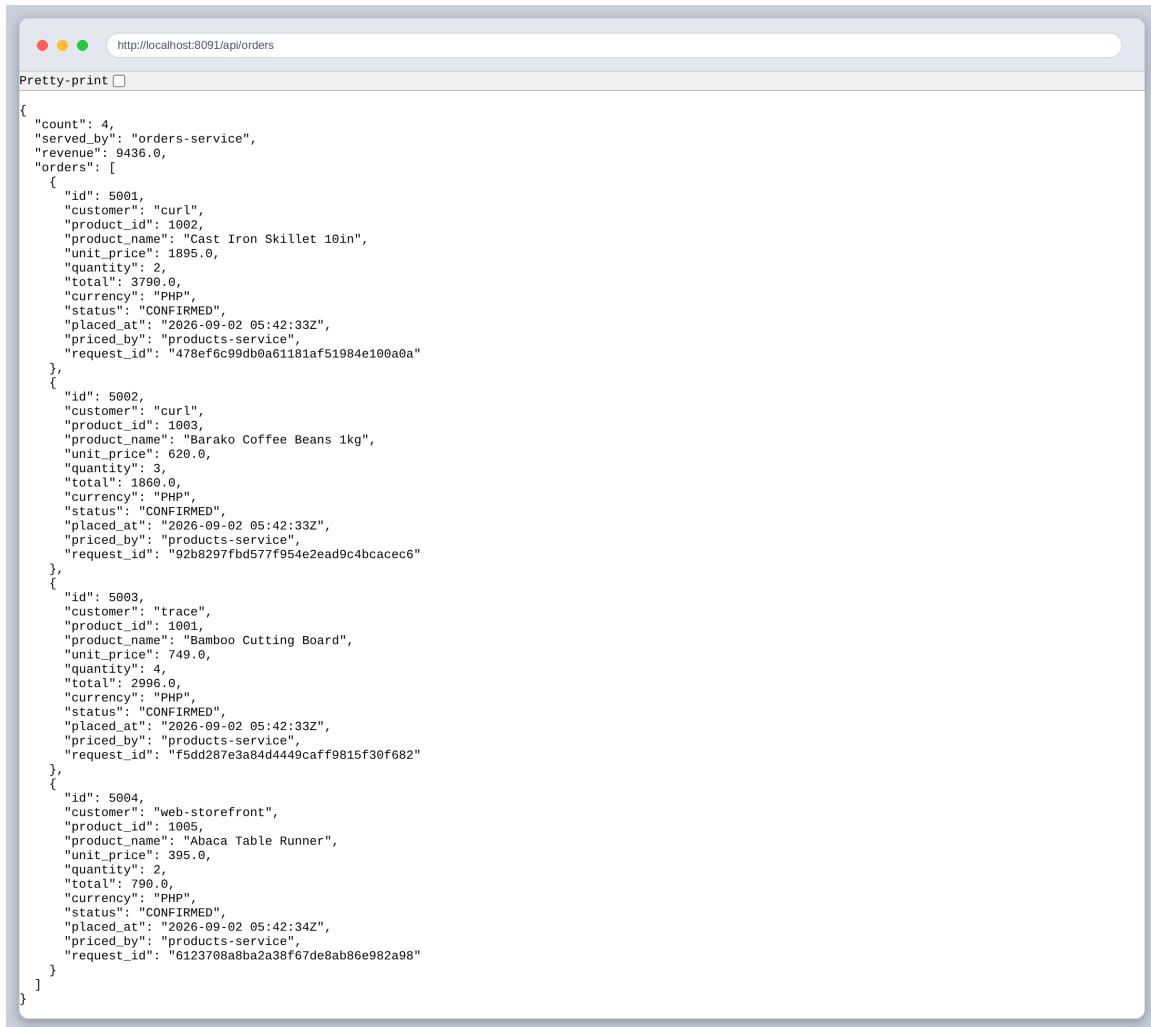


Figure 35: GET /api/products in the browser, through the gateway



```
http://localhost:8091/api/orders

Pretty-print ☐

{
  "count": 4,
  "served_by": "orders-service",
  "revenue": 9436.0,
  "orders": [
    {
      "id": 5001,
      "customer": "curl",
      "product_id": 1002,
      "product_name": "Cast Iron Skillet 10in",
      "unit_price": 1895.0,
      "quantity": 2,
      "total": 3790.0,
      "currency": "PHP",
      "status": "CONFIRMED",
      "placed_at": "2026-09-02 05:42:33Z",
      "priced_by": "products-service",
      "request_id": "478ef6c99db0a61181af51984e100a0a"
    },
    {
      "id": 5002,
      "customer": "curl",
      "product_id": 1003,
      "product_name": "Barako Coffee Beans 1kg",
      "unit_price": 620.0,
      "quantity": 3,
      "total": 1860.0,
      "currency": "PHP",
      "status": "CONFIRMED",
      "placed_at": "2026-09-02 05:42:33Z",
      "priced_by": "products-service",
      "request_id": "92b8297fbd577f954e2ead9c4bcacec6"
    },
    {
      "id": 5003,
      "customer": "trace",
      "product_id": 1001,
      "product_name": "Bamboo Cutting Board",
      "unit_price": 749.0,
      "quantity": 4,
      "total": 2996.0,
      "currency": "PHP",
      "status": "CONFIRMED",
      "placed_at": "2026-09-02 05:42:33Z",
      "priced_by": "products-service",
      "request_id": "f5dd287e3a84d4449caff9815f30f682"
    },
    {
      "id": 5004,
      "customer": "web-storefront",
      "product_id": 1005,
      "product_name": "Abaca Table Runner",
      "unit_price": 395.0,
      "quantity": 2,
      "total": 790.0,
      "currency": "PHP",
      "status": "CONFIRMED",
      "placed_at": "2026-09-02 05:42:34Z",
      "priced_by": "products-service",
      "request_id": "6123708a8ba2a38f67de8ab86e982a98"
    }
  ]
}
```

Figure 36: GET /api/orders. Every order carries the id of the request that created it

Is the gateway really the only way in?

The claim that the services are unreachable is easy to assert and easy to get wrong, so it is tested rather than stated. The same three ports are dialled twice: once on loopback and once on 169.254.255.194, the address another machine would use to reach this one.

```
bash - the same three ports, dialled two ways

=== the same three ports, dialled two ways ===
    loopback    127.0.0.1
    routable    169.254.255.194    (what another machine would use)

PORT    SERVICE                via 127.0.0.1    via 169.254.255.194
-----
8000    products-service        connected        refused
8001    orders-service          connected        refused
8091    api-gateway              connected        connected

=== what that means ===
    Only 8091 answers on the routable address, so the gateway is the
    only way in. The two services are reachable from processes on this
    machine, which includes the gateway, and from nothing else.

    On a runtime where the bridge network works, the same property
    comes from docker-compose.yml instead: the services get `expose`
    and only the gateway gets `ports`.
```

Figure 37: The same three ports, dialled two ways

Only 8091 answers on the routable address. The two services accept connections from processes on this machine, which includes the gateway, and refuse everything else. On a runtime where the bridge network works the same property comes from the compose file instead, with `expose` on the services and ports on the gateway only.

What happens when a service dies

The last test says the most about the design. I stopped the catalogue container and left everything else running.

```
bash - docker stop bazaar-products

=== docker stop bazaar-products ===
$ docker stop bazaar-products
bazaar-products

$ docker ps --format 'table {{.Names}}\t{{.Status}}'
NAMES          STATUS
bazaar-orders   Up 9 seconds
bazaar-gateway  Up 9 seconds

=== the gateway is still up and still answering for itself ===
$ curl -s http://localhost:8091/health
{"service":"api-gateway","status":"up","routes":["/api/products","/api/orders"]}
```

Figure 38: docker stop bazaar-products. Two containers left, and the gateway still answering for itself

```
bash - two different 503s, from two different components

=== but the catalogue route now has nothing behind it ===
NGINX cannot connect, so the shaped 503 from error_page is returned
instead of an HTML error page.
$ curl -s -w '[HTTP %{http_code}]\n' http://localhost:8091/api/products
{"error":"upstream service unavailable","gateway":"nginx-api-gateway","request_id":"dec9b153f71122df92b895dd82f30bc4"}[HTTP 503]

=== and an order cannot be priced ===
orders-service is healthy. It is its dependency that is gone, and
it says so rather than returning a 500.
$ curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' -d '{"product_id":1002,"quantity":1}'
{"detail": {"error": "products service unavailable", "dependency": "http://products-service:8000", "hint": "the catalogue container is not answering, so the order cannot be priced", "reason": "ConnectError"}, "error": "products service unavailable"}[HTTP 503]
```

Figure 39: One 503 from NGINX, one from the orders service

Those two 503s are not the same 503. The first is the gateway's: NGINX could not connect to an upstream at all, so `error_page` produced the shaped JSON from Task 1 instead of an HTML page. The second is the orders service's own, and it is the more useful message, because the orders service is perfectly healthy and it says which dependency is missing, at which address, and why that stops it pricing an order. From those two responses a caller can tell that the catalogue is the problem and the order pipeline is not.

```
bash - what still works with the catalogue gone

=== orders already placed are still readable ===
The orders service holds them; nothing about reading them needs the
catalogue.
$ curl -s -o /dev/null -w 'GET /api/orders -> %{http_code}\n' http://localhost:8091/api/orders
GET /api/orders -> 200

=== and the dependency view shows what it now sees ===
$ curl -s http://localhost:8091/api/orders/dependency
{
  "service": "orders-service",
  "instance": "ip-10-137-53-49.us-east-1.compute.internal",
  "configured_dependency": "http://products-service:8000",
  "dns_name": "products-service",
  "resolved_to": "127.0.0.1",
  "health_probe": {
    "reachable": false,
    "error": "ConnectError"
  }
}
```

Figure 40: Orders already placed are still readable, and the dependency probe reports what it now sees

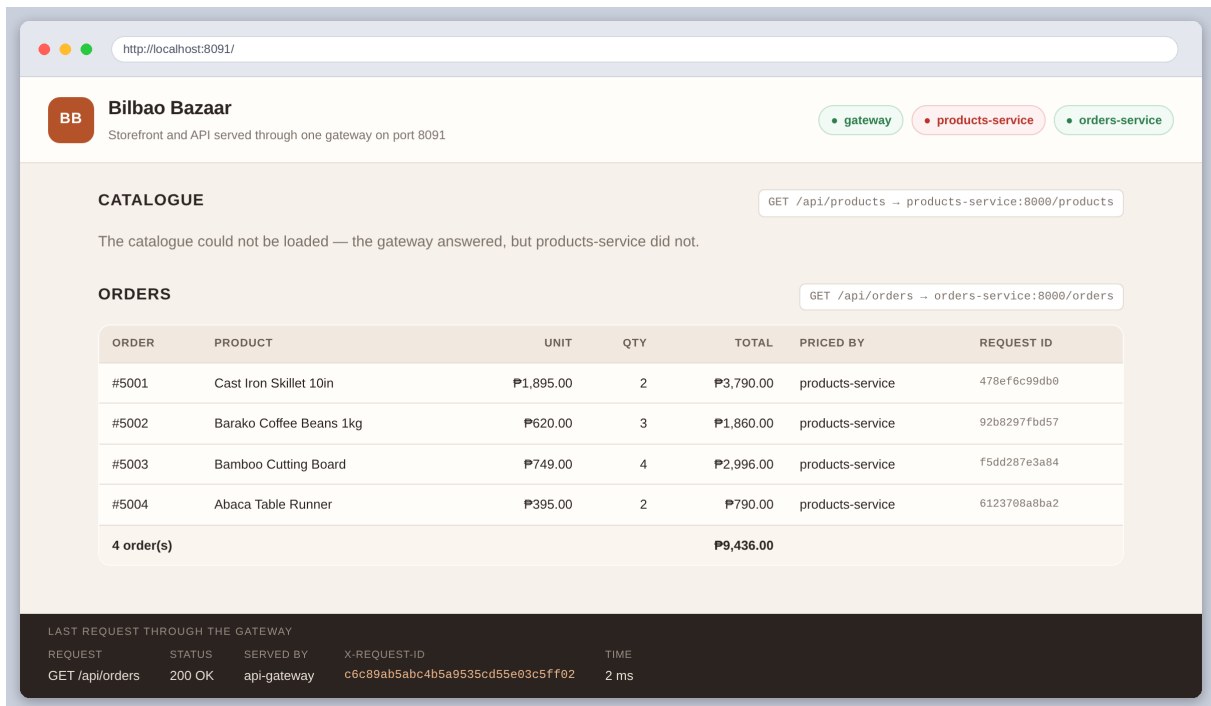


Figure 41: The same page with the catalogue container stopped. The products-service pill has gone red, the catalogue is a message, and the orders table is still there

That screenshot is the clearest thing in this document. The page is not broken, it is partly available, and it is obvious from looking at it which service is down.

```
bash - docker start bazaar-products, and no gateway restart

=== docker start bazaar-products ===
$ docker start bazaar-products
bazaar-products

=== the catalogue route recovers with no gateway restart ===
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8091/api/products
GET /api/products -> 200

=== and orders can be priced again ===
$ curl -s -X POST http://localhost:8091/api/orders -H 'Content-Type: application/json' -d '{"product_id":1007,"quantity":2,"customer":"after-restart"}'
{
  "id": 5005,
  "customer": "after-restart",
  "product_id": 1007,
  "product_name": "Calamansi Marmalade 250g",
  "unit_price": 185.0,
  "quantity": 2,
  "total": 370.0,
  "currency": "PHP",
  "status": "CONFIRMED",
  "placed_at": "2026-09-02 05:42:44Z",
  "priced_by": "products-service",
  "request_id": "b778797196fa8a107e1ea0ee5489dfda"
}

=== NGINX did not need to be reloaded ===
$ docker ps --format 'table {{.Names}}\t{{.Status}}'


| NAMES           | STATUS        |
|-----------------|---------------|
| bazaar-products | Up 1 second   |
| bazaar-orders   | Up 13 seconds |
| bazaar-gateway  | Up 13 seconds |


```

Figure 42: docker start, and the catalogue route recovering with no gateway restart and no reload

BB

Bilbao Bazaar

Storefront and API served through one gateway on port 8091

gateway

products-service

orders-service

CATALOGUE

GET /api/products → products-service:8080/products

KITCHEN

Bamboo Cutting Board

₱749.00

id 1001

24 in stock

1

Place order

KITCHEN

Cast Iron Skillet 10in

₱1,895.00

id 1002

8 in stock

1

Place order

PANTRY

Barako Coffee Beans 1kg

₱620.00

id 1003

52 in stock

1

Place order

HOME

Buri Storage Basket

₱480.00

id 1004

Only 3 left

1

Place order

HOME

Abaca Table Runner

₱395.00

id 1005

17 in stock

1

Place order

KITCHEN

Stainless Tumbler 500ml

₱559.00

id 1006

Out of stock

1

Unavailable

PANTRY

Calamansi Marmalade 250g

₱185.00

id 1007

41 in stock

1

Place order

HOME

Handwoven Placemat Set

₱690.00

id 1008

6 in stock

1

Place order

ORDERS

GET /api/orders → orders-service:8080/orders

ORDER	PRODUCT	UNIT	QTY	TOTAL	PRICED BY	REQUEST ID
#5001	Cast Iron Skillet 10in	₱1,895.00	2	₱3,790.00	products-service	478ef6c99db0
#5002	Barako Coffee Beans 1kg	₱620.00	3	₱1,860.00	products-service	92b8297fbd57
#5003	Bamboo Cutting Board	₱749.00	4	₱2,996.00	products-service	f5dd287e3a84
#5004	Abaca Table Runner	₱395.00	2	₱790.00	products-service	6123708a8ba2
#5005	Calamansi Marmalade 250g	₱185.00	2	₱370.00	products-service	b778797196fa

LAST REQUEST THROUGH THE GATEWAY

REQUEST	STATUS	SERVED BY	X-REQUEST-ID	TIME
GET /api/orders	200 OK	api-gateway	f406c0e45a416286ad0bdd932d4d58fe0	3 ms

Figure 43: The storefront after the catalogue came back. The pill recovers on its own; the poll runs every five seconds

This one deserves a caveat I only understood because of the nginx -t finding earlier. NGINX resolved products-service once, at startup, so recovery worked here because the container came back at the same address. Had it come back somewhere else, the gateway would have kept sending traffic to the old address until it was reloaded. Getting that right needs a resolver directive and the upstream in a variable, and it is the difference between a gateway that survives a restart and one that only appears to.

What went wrong

The application was the quick part. Most of the time went on the container runtime, and the sequence is worth recording because none of it announced itself clearly.

The first version was ordinary: a bridge network, the gateway publishing 8080, the two services on expose so only the gateway could reach them. The gateway would not start, because 8080 was already taken on the machine. I moved it to 8090, and the container started, answered exactly one health check, and died.

From there it got stranger. Podman reported the port as published while nothing was listening on it. Container-to-container calls failed by name, so I tried by address and they failed that way too, which ruled out DNS and pointed at the network itself. Both containers had addresses on the bridge that no packet could reach from anywhere. docker exec refused to run at all, complaining it could not find root in the passwd file. Then podman deadlocked hard enough to need podman system renumber before it would delete a container.

The cause is in podman info, a few lines below where I first looked. The runtime is not rootless, but its capability list has no CAP_NET_ADMIN. Without that, netavark can allocate addresses on a bridge and cannot program any routes to them, so containers get addresses that do not work, and publishing a port fails for the same reason because publishing is NAT. Running podman rootless instead would have given it full capabilities inside a user namespace, and that failed too, with a bare permission denied.

The deadlock had a separate cause worth knowing: podman-compose puts every service in a pod, so the containers shared an infra container's network namespace, and that combination is what wedged the runtime. So docker-compose.yml is validated with docker compose config and is the declarative description of the system, but run.sh starts the containers with explicit docker run commands carrying the same environment. I would rather ship a compose file that says what the system is and be honest that the evidence came from the equivalent commands.

Host networking was the way out: all three containers share the host network namespace, the services bind loopback, the gateway binds everything. The isolation test earlier is what convinced me the property I wanted had actually survived the change.

One more, which cost the most time for the least reason. After all that, the gateway was demonstrably listening on 8090 in the right namespace and no connection to it ever completed. A plain Python HTTP server on the same port behaved identically, which finally removed NGINX from suspicion. Port 8090 is filtered somewhere in that sandbox: a process binds it happily and nothing ever arrives. 8091 works. Nothing was wrong with the configuration at all.

The lesson I will actually keep: when a component looks broken, replace it with the most boring possible substitute before reading its configuration again. One `python3 -m http.server` would have saved me an hour if I had reached for it first, instead of assuming that a web server failing to serve was a web server problem.

Bugs in my own work

Three, all found by looking at output rather than by reasoning about code.

- The X-Request-ID came back twice. The gateway adds it and the services echo it, and the storefront duly displayed the identifier, a comma, and the same identifier again, because fetch joins repeated headers. proxy_hide_header drops the upstream copy so only the one from the component that minted it leaves.
- /health returned no X-Request-ID while every other route had one. That is the add_header inheritance rule described in Task 1, and I would not have noticed if the health endpoint had not been in the same table of captured headers as the proxied routes.
- Successful responses were indented and error responses were not, because FastAPI's default_response_class applies to routes and not to exception handlers. Two figures side by side disagreeing about formatting is what made me look.

Conclusion

The routing was the smallest part of this. Two upstream blocks and four location blocks and the gateway does what the assignment asked, and I had that working before I understood anything interesting.

What the gateway is actually worth showed up around the routing. The storefront needs no CORS configuration because the page and the API share an origin, and that is work I never had to do rather than a feature I can point at. A stopped container produces a JSON 503 instead of an HTML error page because of four lines near the bottom of the file. Every request gets an identifier that makes it traceable across three containers, and no service had to be taught to generate one.

On the service-to-service side, the thing I would carry into a bigger system is that most of the code in the east-west call is error translation, and that this is correct rather than defensive clutter. The outage test proved it: an orders service returning a 503 that names its missing dependency is genuinely more useful than a 500, and it cost about ten lines. Choosing REST over a queue was right here for a boring reason, which is that the caller cannot proceed without the answer, and I would rather be able to say why than have reached for a broker because the assignment mentioned one.

The rest of what I learned was about being wrong efficiently. A build failing on a valid config file taught me when NGINX resolves upstream names, which then explained both why compose needs depends_on and why the recovery test only worked because the container came back at the same address. An hour lost to a filtered port taught me to substitute the simplest possible component before rereading configuration. Both came from things not working, which is not the part of an assignment I would have chosen and is most of what I took away from it.